OBJECT ORIENTED PROGRAMMING SYSTEM

OOPs is a programming system used to represent real-world objects in the software
world using classes and objects.

why OOP's

*** To Store Data in Organized Manner**

*** To Access Data Easily**

*** Code Reusability**

*** Code Maintenance**

Class

Class is a blueprint / plan / model / design
used to create objects.

Object

**Object is a real-world entity created from a
class that contains attributes
(data) and methods (behavior).**

SYNTAX TO CREATE CLASS

```
class ClassName:
     Variables (Attributes)
    # → Used to store data (properties)

    Constructor
    # → Special method used to
initialize object automatically

    Methods
    # → Functions inside the class
```

*If we want to access any member of a class from outside the class, we can do it in two ways:

1. By using class name
2. By creating an object

*syntax to create object,

```
    var_name = ClassName()
```

*What is the Reference variable---->???

The variable that can be used to refer to an object is called a reference variable.

note:

--->>1st letter of each word in class name should start with capital letter.

```
class Employee:          #Employee ---> class name
     eid = 420            #   }
     ename = "Kasab"      #   }  ---> class variable (attributes)
     gender = "Male"      #   }
```

```
# print(eid)       #NameError: name 'eid' is not defined. Did you mean: 'id'?
```

#1.accessing class variable by using class name

```
# print(Employee.eid)     #420
```

```
# print(Employee.ename)   #Kasab
# print(Employee.gender)  #Male


Note:---> if we didn't mention the bracket it
will point to the main class only.

# e = Employee          #assigning class name to
other variable

# print(e)          #<class '__main__.Employee'>


Note:--> if we mention the bracket it will
pointing to the main class object address
# e = Employee()        #object creation

# print(e)          #<__main__.Employee object at
0x0000021A57047C70>


#2.accessing class variable by using object

# e = Employee()
# print(e.ename)        #Kasab
# print(e.eid)          #420
# print(e.gender)       #Male
```

```
class Demo:
    a=10
    b=20

d=Demo()          #-----> object1
d1=Demo()        #------> object2


print(Demo.__dict__)

{'__module__': '__main__', 'a': 10, 'b': 20,
'__dict__': <attribute '__dict__' of 'Demo'
objects>, '__weakref__': <attribute
'__weakref__' of 'Demo' objects>, '__doc__':
None}
```

KEY POINTs

1. Class members are stored in a dictionary-like structure called namespace (`__dict__`).

2. In this dictionary:

- **Member name acts as the key**
- **Member value acts as the value**

3. When an object is created, separate memory is allocated for each object.

4. Each object stores its own data (instance variables) independently.

5. Objects refer to the class for accessing class variables (shared data).

6. Each object has its own independent memory, but shares common data from the class.

Extra one:----> optional

documentation string or doc-string
****
>Documentation string represents description of the class

>within the class doc string is always optional

>we can get doc string by using the following 2 ways

1.print(Classname.__doc__)

2.help(Classname)

```
by using Classname.__doc__
*******************************

class IPL:
    """Details about my team"""
    team="RCB"
    player="Virat"
    j_no=18

print(IPL.__doc__)   #Details about my team
i=IPL()


by using help(Classname)
*******************************

class IPL:
    """Details about my team"""
    team="RCB"
    player="Virat"
    j_no=18

help(IPL)
i=IPL()

#out-put:--->
"""
Help on class IPL in module __main__:

class IPL(builtins.object)
 |  Details about my team
```

```
 |
 |  Data descriptors defined here:
 |
 |  __dict__
 |      dictionary for instance variables (if
defined)
 |
 |  __weakref__
 |      list of weak references to the object (if
defined)
 |
 |
-----------------------------------------------
---------------------
 |  Data and other attributes defined here:
 |
 |  j_no = 18
 |
 |  player = 'Virat'
 |
 |  team = 'RCB'
```

class variable modification by class name and by object:

****

Note:--->

*any modification done by class will impact all other objects.

*any modification done by object it will not impact to class and other object and also connection between class and
object will be lost .

*1st modification done by object and then modification done by class, the changes done by class are not impacting the object.

```
# class Demo:
#     a = 10
#     b = 20

# d1 = Demo()      #object1
# d2 = Demo()     #object2


#way1: modification  by class name

#before modification

# print(Demo.a)           #10
# print(d1.a)             #10
# print(d2.a)             #10

#after modification
```

```
# Demo.a = 100
# print(Demo.a)           #100
# print(d1.a)             #100
# print(d2.a)             #100


#way2: modification done by object1 --> d1

# d1.a = 111
# print(Demo.a)           #10
# print(d1.a)             #111
# print(d2.a)             #10



#modification done by object1 --> d2

# d2.b = 222
# print(Demo.b)           #20
# print(d1.b)             #20
# print(d2.b)             #222



#way3: modification done by object then again
modification done by class

#modification done by object -> d1
# d1.a = 1010
# print(Demo.a)           #10
```

```
# print(d1.a)                #1010
# print(d2.a)                #10


#modification by class
# Demo.a = 1001
# print(Demo.a)              #1001
# print(d1.a)                #1010
# print(d2.a)                #1001


#way4: modification done by class then again
modification done by object

#modification by class
# Demo.b = 2002
# print(Demo.b)              #2002
# print(d1.b)                #2002
# print(d2.b)                #2002


#modification done by object -> d1
# d1.b = 2020
# print(Demo.b)              #2002
# print(d1.b)                #2020
# print(d2.b)                #2002
```

Methods

In simple words a function inside a class is called as method

Types of methods:--->
1.instance method
2.class method
3.static method

1.instance method

* instance means object.

*any method which accept object address as an argument is called as instance method

OR

Instance method is a method it will work on object data

*All instance method 1st argument will always pointing to object address/will accept object as an argument

*when ever we call any instance method with object,no need to pass object address explicitly,because
implicitly the object will be passed.

***If we pass object address explicitly then it will throw error**

*Instance method 1st arguments name that is pointing to object can be named with anything but as per PEP8 standard
it is better to write "self" as an argument name.

*Whenever we call an instance method with class name then explicitly we need to pass object address.

Key Points:

1. The first parameter is always `self` (refers to the current object).

```
syntax:--->
class Classname:
     def function_name(self):
          ...

object=Classname()


calling by object:-->
object.function_name()


calling by class name:-->
Classname.function_name(object)

"""
```

Example 01

```
class Greeting:
    def say_hello(self, name):
```

```
        print(f"Hello, {name}! Welcome
to Python OOP.")

g1 = Greeting()

g1.say_hello("Prince")
Greeting.say_hello(g1, "Raj")
```

Example 02
Using Class Variable in Instance Method

```
class Animal:
    species = "Dog"    # class variable
    def sound(self):
        print(f"{self.species}
barks!")
# Object
a1 = Animal()
a1.sound()
An imal.sound(a1)    # explicit object
```

Example 03

```
#instance method without argument

class Employee:
    def sal_cal(self):
        sal=8000
        print(your total salary is {sal}')

e=Employee()
#calling through object
e.sal_cal()          #your total salary is 8000

#calling through classname
Employee.sal_cal(e)   #your total salary is 8000
```

Example 04

```
# instance method with argument

#wap to give 1000 increment to all the salary

class Employee:
    def sal_cal(self,sal):
        print(monthly salary is Rs{sal},with 1000
OT total sal is RS{sal+1000}')


e=Employee()
e.sal_cal(7500)    #monthly salary is Rs7500,with
1000 OT total sal is RS8500

Employee.sal_cal(e,7500)   #monthly salary is
Rs7500,with 1000 OT total sal is RS8500
```

Example 05

accessing class variable inside the instance method:

*if we want to access any class variable inside the instance method then we can access by class name (or)
by object that is "self" parameter.

```
*self --> it is a special variable pointing to
current object address.
"""

#wp to print ordered product and delivery product
in amazon class
"""
class Amazon:
    pname = "watch"
    price = 2300
    cname = "Amith"
    caddr = "#123 Banglore-561900"

    def orderlist(self):
        print(f"Product {pname} is ordered the
price is Rs.{price}")

    def delivery_product(self):
        print(f"Customer{cname} is ordered a
product {pname} and delevring to {caddr}")

a = Amazon()
a.orderlist()
a.delivery_product()

#o/p: NameError: name 'pname' is not defined

#as per the above program we will get name error,
bcz when we want access any class member we
should access by class name
```

```
and object
"""



#wp of accessing class variable inside method by
class name

class Amazon:
    pname = "watch"
    price = 2300
    cname = "Smith"
    caddr = "#123 Banglore-561900"

    def orderlist(self):
        print(f"Product {Amazon.pname} is ordered
the price is Rs.{Amazon.price}")

    def delivery_product(self):
        print(f"Customer {Amazon.cname} is
ordered a product {Amazon.pname} and delivering
to {Amazon.caddr}")

# a = Amazon()
# a.orderlist()
# a.delivery_product()
#accessing class variable inside the method by
class name and modifying by class name

class Qsp:
    rating = "1*"
```

```
    sub = "SQL"

    def display_mock_rating(self):
        print(f"ur mock rating is {Qsp.rating} in
{Qsp.sub} in subject")

# q = Qsp()
# q.display_mock_rating()
#ur mock rating is 1* in SQL in subject

# q.sub = "MANUAL"
# q.rating = "2"
# q.display_mock_rating()
#ur mock rating is 1* in SQL in subject


"""
*as per the above program we are expecting that
o/p will be "ur mock rating is 2 in MANUL
subject", but we got
the same old o/p.

*because inside the instance method we are
accessing by class name, and modification done by
object, any modification
done by object will not impact on class level.


*so it is a better practice in an instance method
calling a class variable with "object" this
"self".
```

```
"""


#accessing class variable inside the method by
class name and modifying by object

class Qsp:
    rating = "1*"
    sub = "SQL"

    def display_mock_rating(self):
        print(f"ur mock rating is {self.rating}
in {self.sub} in subject")

  q = Qsp()
  q.display_mock_rating()
  ur mock rating is 1* in SQL in subject

  Qsp.sub = "Apti"
  Qsp.rating = "1"
  q.display_mock_rating()
  ur mock rating is 1 in Apti in subject

  q.sub = "MANUAL"
  q.rating = "2"
  q.display_mock_rating()
 ur mock rating is 2 in MANUL in subject

  Qsp.sub = "Python"
  Qsp.rating = "3"
  q.display_mock_rating()
```

ur mock rating is 2 in MANUL in subject

Instance Method Examples (Practice_example)

1.Create a class Person
Store name and age
Display details

2.Create a class Dog
Store name and breed
Print dog details

3.Create a class Fan
Store brand and price
Display details

4.Create a class Employee
Store name and salary
Increase salary by 5000
Display details

5.Create a class Gameplayer
Store player name and score
Increase score

Display score

6. BANK CLASS
Question:
WAP TO CREATE A CLASS NAME AS A BANK
Create a class Bank:
Create account (name, balance)
Deposit money
Withdraw money
Display balance

7. STUDENT CLASS
Question:
Create a class Student:
Store marks
Calculate grade

8.MOBILE CLASS
Question:
Create a class Mobile:
Store brand & price
Update price
Display details

9. ACCOUNT CLASS
Question:

Create a class Account:
Store account number & balance
Check balance

10.Create a class Marks
Store marks of 3 subjects
Calculate total and average

class method

- Any method decorated with `@classmethod` is called a **class method.**
- Class methods can be called using the **class name** (preferred) or via an **instance.**

The **first argument** of a class method receives the **class object** (not the instance). By convention that parameter is named `cls`.

- Class methods are **bound to the class,** so they typically operate on **class-level data** (class variables) or provide **alternative constructors** .

```
syntax:
-------
class classname
    @classmethod
    def fun(cls, arg1, arg2, ...):
        ----

classname.func()
     (or)
obj.func()
```

Why use class methods?

- Alternative constructors
- Operate on or modify class variables shared across instances.

Important note Point

- Whether you call the method on the **class** or on an **instance**, Python **automatically passes the class object** as the first argument —

you must **not** pass it manually.

- The value passed is the **class object** (e.g., `<class '__main__.Sample'>`), not the textual class name.
- Although you *can* name the first parameter anything, **PEP8 recommends `cls`**. Using `self` for a classmethod is allowed by the interpreter but **strongly discouraged** because it is confusing (readers expect `self` → instance).
- Class methods cannot directly access instance attributes (unless you explicitly receive or create an instance inside the method).

Example 01

```
class Sample:
    @classmethod
    def add(cls):
```

```
        print("I'm a class method")
        print(cls)

Sample.add()
# I'm a class method
# <class '__main__.Sample'>

s = Sample()
s.add()
# I'm a class method
# <class '__main__.Sample'>
```

Example 02

```
class Demo:
```

```
    @classmethod
    def show(cls):
        print("This is a class method")
# Calling by object
d = Demo()
d.show()
Calling by className
Demo.show()
```

Example 03

```
Accessing Class Variable
class School:
    school_name = "ABC Public School"
    @classmethod
    def display_school(cls):
        print("School Name:", cls.school_name)
School.display_school() #Before modification
s1 = School()
```

```
s1.display_school()
```

Static Method

- A **static method** is a type of method in a class that does **not take `self` (object reference) or `cls` (class reference)** as its first parameter.
- It behaves just like a **normal function,** but it belongs to a class's namespace.
- It is defined using the `@staticmethod` decorator.

KEY POINTS

A static method **does not depend on the class or instance** — it is independent.

It is useful when you want to write utility/helper methods inside a class.

You can call a static method using:

- The **class name**
- The **object reference**
  (but no extra implicit arguments are passed).

Static methods can accept arguments, but those must be passed **explicitly** by the caller.

Unlike **instance methods** and **class methods**, static methods **cannot access or modify class-level or instance-level attributes** directly.

syntax:

```
-------
class ClassName:
    @staticmethod
    def fun1():
        ...
ClassName.fun1()
    (or)
```

```
c = ClassName()
c.fun1()
```

Example 01

```
class MathUtils:
    @staticmethod
    def add(a, b):
        return a + b

    @staticmethod
    def greet(name):
        print(f"Hello, {name}!")

# Call using class name
print(MathUtils.add(10, 20))    # 30
MathUtils.greet("Prince")       # Hello, Prince!

# Call using object
```

```
obj = MathUtils()

print(obj.add(5, 7))            # 12

obj.greet("Gouda")              # Hello, Gouda!
```

Example 02

```
class Demo:
    @staticmethod
    def add():
        print("im a static method")

#accessing static method by object

# d = Demo()
# d.add()          #im a static method

#accessing static method by class name
# Demo.add()       #im a static method

class Calculator:
    a = 10
    b = 20
    @staticmethod
    def add(a, b):
        return a+b
    @staticmethod
    def sub(a, b):
```

```
        return a - b
    @staticmethod
    def mul(a, b):
        return a * b
    @staticmethod
    def div(a, b):
        return a/b

# c = Calculator()
# print(c.add(10, 5))                #15
# #modification
# c.a = 100
# Calcultor.b = 200
# print(c.add(10, 5))                #15


"""
*in the above example we are modifying by class
name and object, generally we will think output
will be 300, but still
the output will be 15, because any modification
done will not impact the static method.
*because it is not associated with class/object.
"""
```

Quick Comparison Table

Feature	Instance Method	Class Method	Static Method
Decorator	None	@classmethod	@staticmethod
First Argument	self (object)	cls (class)	No default argument
Access	Instance + Class variables	Class variables only	None (independent)
Usage	Works with **object data**	Works with **class data**	Utility/helper functions
Call By	Object (preferred)	Class or Object	Class or Object

constructor:

****

*It is a special member of class.

*in python constructor will be defined by __init__() magic method

*constructor will get executed whenever we create an object/instance.

*constructors are used to initialize/instantiate instance variables.

(in simple words to declare instance variable)

*we call constructor in 2 ways,

1.manually calling by class name
syntax: classname.__init__(object)

2. when we create object
syntax: var_name = classname() ===> internally calling __init__()

Types of Constructors in Python

1. Default Constructor

A constructor without parameters (except self)

```
class Student:
    def __init__(self):
        print("Default Constructor")

s1 = Student()
```

2. Parameterized Constructor

A constructor with parameters

```
class Student:
    def __init__(self, name, age):
```

```
        self.name = name
        self.age = age

    def display(self):
        print(self.name, self.age)

s1 = Student("Prince", 22)
s1.display()
```

```
#example on constructor without any argument

class Animal:
    def __init__(self):
        print("Welcome to Zoo all you :)))")

    def sound(self):
        print(f"Dog says BowBow")
```

```
# a = Animal()          #object creations
internally __init__() will get execute
# a.sound()
#Welcome to Zoo all you :)))
#Dog says BowBow


#calling constructor manually
#Animal.__init__(a)
#Welcome to Zoo all you :)))
```

EXamples

```
class Demo:
    def __init__(self):
        print("Constructor is
called")
```

```
d1 = Demo()
```

Ex2

```
class Student:
    def __init__(self):
        self.name = "Prabhu"

s1 = Student()
print(s1.name)
```

Ex3

```
class Student:
   def __init__(self):
      self.name = "Ram"
      self.age = 21

s1 = Student()
```

```
print(s1.name, s1.age)
```

Parameterized Constructor (Examples)

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

s1 = Student("Anu", 20)
print(s1.name, s1.age)
```

Ex5

```
class Mobile:
    def __init__(self, brand, price):
        self.brand = brand
        self.price = price

m1 = Mobile("Samsung", 20000)
m2 = Mobile("Apple", 80000)
```

```
print(m1.brand, m1.price)
print(m2.brand, m2.price)
```

Constructor + Method Together

```
class Student:
    def __init__(self, name):
        self.name = name

    def greet(self):
```

```
        print("Hello", self.name)

s1 = Student("Rahul")
s1.greet()
```

Ex 6

```
class Bank:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

    def show(self):
        print(self.name, "has", self.balance)

b1 = Bank("Ravi", 5000)
b1.show()
```

Constructor with Calculation

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
        self.area = length * width   # calculation inside
constructor

r1 = Rectangle(5, 3)
print("Area:", r1.area)
```

Output :- 15

Constructor + Condition Logic

```
class Student:
    def __init__(self, marks):
        self.marks = marks
        if marks >= 40:
            self.result = "Pass"
        else:
            self.result = "Fail"

s1 = Student(35)
print(s1.result)
```

Constructor Calling Another Method

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
        self.display()   # calling method inside
constructor

    def display(self):
        print(self.name, self.marks)

s1 = Student("Anu", 90)
```

Class Variable + Constructor

```
class Student:
    college = "ABC College"   # class variable

    def __init__(self, name):
        self.name = name      # instance variable

s1 = Student("Ram")
s2 = Student("Anu")

print(s1.name, s1.college)
```

```
print(s2.name, s2.college)
```

method overloading:

```
*In python we can't achieve method overloading, because python is a dynamic programming language.

*definition: developing a multiple methods with a same name but variation in argument list, variation in argument list means,
   *variation in length of argument
   *variation in data type
   *variation in order of occurrence of data type.

*In python we can achieve method overloading partially by using "default argument".
"""

"""
```

```
class Sample:
   def add(self, a):
       print("addition1")
       print(a)

   def add(self, a, b):
       print("addition2")
       print(a+b)

   def add(self, a, b, c):
       print("addition3")
       print(a+b+c)
"""
# s = Sample()

"""
s.add(10)
TypeError: Sample.add() missing 2 required
positional arguments: 'b' and 'c'
s.add(10, 20)
TypeError: Sample.add() missing 1 required
positional argument: 'c'

according to above calling method that is add(),
we thought if we pass single argument add(a) will
get execute
but python is a dynamic programming language, it
will consider the latest method that is
add(a,b,c).
"""
# s.add(10, 20, 30)
```

```
# addition3
# 60

class Sample:
    def add(self, a):
        print("addition1")
        print(a)

    def add(self, a, b):
        print("addition2")
        print(a+b)

    def add(self, a=0, b=0, c=0):
        print("addition3")
        print(a+b+c)
# s = Sample()
# s.add(10)
# s.add(10, 20)
# s.add(10, 20, 30)

"""
addition3
10
addition3
30
addition3
60
```

constructor overloading:

****

```
*developing multiple constructors in the
same class with variation in argument list
is called constructor overloading.
"""
"""
class Dim:
    def __init__(self):
        print("constructor1")

    def __init__(self, a):
        print("constructor2")

    def __init__(self,a ,b):
        print("constructor3")

d = Dim(10, 20)          #constructor3
"""

class Dim:
    def __init__(self):
        print("constructor1")

    def __init__(self, a):
        print("constructor2")
```

```
    def __init__(self,a=0, b=0):
        print("constructor3")
        print(a+b)


# d = Dim(10)
#constructor3
# 10
```

Constructor with variable number of arguments

If we pass more than declared variable values to _ *init_() it will Throw error to avoid this we use *args*

```
class Car1:
    def
__init__(self,brand,model,price,*args):
        print(f'{brand} brand with {model}
price is {price}')
        print(f'extra information are
{args}')

c=Car1("kia","k7","Rs151kh")
c1=Car1("kia","k7","Rs151kh","white","2019
engine")
```

Assignment Examples

1. Store and Display Name

Create a class `Student`

Constructor should take `name`

Print the name

Your Task
Output: Rahul

2. Store Two Values

Create class `Employee`

Take `name` and `salary`

Print both

Output: Ravi 25000

3.Calculate Square

Constructor takes a number
Store square in variable
Print result

Input: 5
Output: 25

4. Create class `Laptop`
Store:

- **brand**
- **price**
- **RAM**

Print like:

HP 50000 16GB

inheritance (IS-A Relationship)

```
*******************************
*a class will inherit a property from one class
to another class is called inheritance.
                                (or)
*a child class inherits a property from the
parent class is called inheritance.

*inheritance is classified into 5 types,
1.single level inheritance
2.multi level inheritance
3.multiple inheritance
4.hierarchical inheritance
5.hybrid inheritance.
"""
"""
```

1.single level inheritance:

```
-------------------------
*A single child class will inherit a property
from a single parent class is called single level
inheritance.
diagram:

"""
class Parent1:
    bal = "3Lac"
    def house(self):
        print("parent house")
```

```
class Child1(Parent1):
   def studying(self):
       print("child im still studying")

# c = Child1()
# print(c.bal)
# c.house()
# c.studying()

# 3Lac
# parent house
# child im still studying

# print(Parent1.__dict__)
# # {'__module__': '__main__', 'bal': '3Lac',
'house': <function Parent1.house at
0x000001EBAC168040>,
'__dict__': <attribute '__dict__' of 'Parent1'
objects>, '__weakref__': <attribute
'__weakref__' of 'Parent1' objects>, '__doc__':
None}
# print(dir(Child1))
# # ['__class__', '__delattr__', '__dict__',
'bal', 'house', 'studying']
```

Examples

```
class Dad:
    def spam(self):
        print("Dad's class")
class Child(Dad):
    def demo(self):
        print("Child class")
x=Child()
x.demo()
x.spam()
Child.spam(x)
Child.demo(x)
```

```
class Dad:
    Agree_land=100
    def villa(self):
```

```
        print("Dad's villa")

class Child(Dad):
    name="Joy"
    def Study(self):
        print("No study")
x=Child()
print(x.Agree_land)
x.villa()
print(x.name)
x.Study()
# print(dir(Dad))
# print()
# print(dir(Child))


class Dad:
```

```
    def show(self):
        print("method-->1")
class Child(Dad):
    def show(self):
        super().show()
        print("method-->2")
s=Child()
s.show()

class Parent:
    def __init__(self):
        print("constructor
class 1")

class Child(Parent):
    def __init__(self):
        super().__init__()
```

```
        print("constructor
class 2")

c=Child()
```

```
class Student:
defstudent_data(self,name,s
ub):
        self.name=name
        self.sub=sub
        print(f'StudentName
is {self.name}')
        print(f'SUbject Name
is {self.sub}')
```

```
class Information(Student):
defstudent_data(self,clg_na
me,marks):

super().student_data(name="
XYZ",sub="SQL")
        self.clg=clg_name
        self.marks=marks
        print(f'CLG Name is
{self.clg}')
        print(f'Total Marks
is {self.marks}')

i=Information()
i.student_data("PYSPIDERS",
99)
```

```
'''
```

Assignment Questions

1.Create a class `Employee` with attributes `name` and `salary`.
Create a child class `Manager` that prints both values.
2.Create a class `Bank` with attribute `balance`.
Create a child class `SavingsAccount` with methods:

- **deposit()**
- **withdraw()**

3.Create a class `Person` with a constructor.
Create a child class `Student` and use `super()` to initialize parent data.

4.Create a class `Vehicle` with method `start()`. Override this method in child class `Bike`.

2.multi level inheritance:

****

*child class inheriting property from its parent class and intern parent class inheriting property from its parent class is called multi level inheritance.

Example 01

```
class Grandpa:
   def land(self):
       print("agriculture land")
# print(dir(Grandpa))  #     ['land']]

class Father(Grandpa):
```

```
    def car(self):
        print("my car")
# print(dir(Father))  #     [ 'car', 'land']]

class Child(Father):
    def loan(self):
        print("lot of loans")
# print(dir(Father))  #     ['loan', 'car',
'land']]

# c = Child()
# c.loan()
# c.car()
# c.land()
```

Example 02

```
class University:
    USN = "1PY12012023"

class Collage(University):
    def exam(self):
        print("lab exam")
    def internal(self):
        print("internal exam")

class Student(Collage):
    def assignment(self):
```

```
        print("copying assignment last moment")

# s = Student()
# print(s.USN)
# s.internal()
# s.assignment()
# s.exam()
```

Example 03

```
class GrandParent:
    def Hello(self):
        print("Hello from GrandParent")
class Parent(GrandParent):
    def Hello_method(self):
        print("Hello from Parent")
class Child(Parent):
    def Hello_child(self):
        print("Hello from child")


c=Child()
c.Hello()
```

```
c.Hello_method()
c.Hello_child()
```

Example 04

```
#method overriding in method------> super().method_name()
class GrandParent:
    def Villa(self):
        print("GrandParent Property")
class Parent(GrandParent):
    def Villa(self):
        print("Parent Property")
        super().Villa()
class Child(Parent):
    def Villa(self):
        print("Child Property")
        super().Villa()
c=Child()
c.Villa()
```

Example 05

```
class Dad:
    def __init__(self):
        print("Gold property")
class MoM(Dad):
    def __init__(self):
        print("siliver Property")
        super().__init__()

class Child(MoM):
    def __init__(self):
        print("Money----loading")
        super().__init__()

x=Child()
```

Example 06

```
class Person:
    def __init__(self,name,age):
        self.name=name
        self.age=age
    def show(self):
        print(f'my name is {self.name} and mine age is {self.age}')
class Teacher(Person):
    def __init__(self,name,age,subject):
        super().__init__(name,age)
        self.sub=subject
    def show_data(self):
        print(f'mine subject name is {self.sub}')


class Incharge(Teacher):
    def __init__(self,name,age,subject,class_name):
        super().__init__(name,age,subject)
        self.class_name=class_name


    def details(self):
        print(f'i am taking in charge of {self.class_name}')
```

```
i=Incharge("Meera",28,"Python","A11")
i.show_data()
i.details()
i.show()

print("**********************************************************")
y=Incharge("Avii",26,"PowerApps","A18")
y.show_data()
y.details()
y.show()



print("-----------------------------------------------------------------")

class Person:
    def __init__(self,name,age):
        self.name=name
        self.age=age
    def show(self):
        print('my name is {self.name} and mine age is {self.age}')
class Teacher(Person):
```

```
    def __init__(self,subject):
        super().__init__(name="Meera",age=28)
        self.sub=subject
    def show_data(self):
        print(f'mine subject name is {self.sub}')


class Incharge(Teacher):
    def __init__(self,class_name):
        super().__init__(subject="Python")
        self.class_name=class_name


    def details(self):
        print(f'i am taking in charge of {self.class_name}')


i=Incharge("A11")
i.show_data()
i.details()
i.show()
print("second object------------>> Data")
x=Incharge("M32")
x.show_data()
x.details()
```

```
x.show()
```

Example 07

```
class BankAccout:
    def __init__(self,Holder,Balance):
        self.Holder=Holder
        self.bal=Balance
    def Data(self):
        print(f'Account Holder Name : {self.Holder} '
              f'and Total balance :{self.bal}')
class Saving(BankAccout):
    def __init__(self,Holder,Balance,Interset_rate):
        super().__init__(Holder,Balance)
        self.rate=Interset_rate

    def Add_interset(self):
        self.bal+=self.bal*self.rate/100
        print(f'after applying the interset total balance is {self.bal}')

class SeniorSaving(Saving):
```

```
    def __init__(self,Holder,Balance,Interset_rate,age):
        super().__init__(Holder,Balance,Interset_rate)
        self.age=age

    def Age_benfits(self):
        if self.age>=60:
            self.bal+=self.bal*1/100
            print(f'after applying 1% interset total amount is {self.bal}')

x=SeniorSaving("Pavan",10000,5,65)
x.Data()
x.Add_interset()
x.Age_benfits()

print()
print("age is less than 60------->")

y=SeniorSaving("Pavan",10000,5,59)
y.Data()
y.Add_interset()
y.Age_benfits()
```

3.multiple inheritance:

****

*A single child class inheriting property from multiple parent classes is called multiple inheritance.

Example 01

```
class Father:
    def quality(self):
        print("Father is hardworking ")

class Mother:
    def skill(self):
        print("Mother is intelligent ")

class Child(Father, Mother):
    def own_talent(self):
        print("Child is creative ")
```

```
# Create Child object
c = Child()
c.quality()       # from Father
c.skill()         # from Mother
c.own_talent()    # from Child
```

Example 02

```
class A:
    def display(self):
        print("Display from A")
        super().display()   # goes to B next

class B:
    def display(self):
        print("Display from B")
```

```
class C(A, B):
    def display(self):
        print("Display from C")
        super().display()   # goes to A

obj = C()
obj.display()
```

Example 03

```
class X:
    def action(self):
        print("Action from X")
        super().action()

class Y:
    def action(self):
        print("Action from Y")
```

```
class Z(X, Y):
    def action(self):
        print("Action from Z")
        super().action()

z = Z()
z.action()
```

Example 04

```
class Parent1:
    def __init__(self):
        print("Parent1 Constructor")

class Parent2:
    def __init__(self):
```

```
        print("Parent2 Constructor")


class Child(Parent1, Parent2):
    def __init__(self):
        print("Child Constructor")
        super().__init__()   # follows MRO


c = Child()
```

Example 05

Constructor & Method Overriding with Parameters (Using `super()`)

```
class Parent1:
    def __init__(self, name):
        print(f"Parent1 Constructor: Hello {name}")

    def show(self, age):
```

```
        print(f"Parent1 Show: Age is {age}")


class Parent2:
    def __init__(self, city):
        print(f"Parent2 Constructor: You live in {city}")


    def show(self, country):
        print(f"Parent2 Show: Country is {country}")


class Child(Parent1, Parent2):
    def __init__(self, name, city):
        print("Child Constructor starts")
        super().__init__(name)    # Calls Parent1 first (MRO)
        Parent2.__init__(self, city)  # Manually call second
parent
        print("Child Constructor ends")


    def show(self, age, country):
        print("Child Show Method")
```

```
        super().show(age)             # Calls Parent1's show
        Parent2.show(self, country)   # Manually call Parent2's
show


# Test
c = Child("Prince", "Bangalore")
c.show(25, "India")
```

Example 06

```
class Person:
    def __init__(self, name, age):
        print(f"Person Constructor: Name={name}, Age={age}")

    def details(self, city):
        print(f"Person Method: City = {city}")

class Company:
```

```
    def __init__(self, company_name, salary):
        print(f"Company Constructor: Company={company_name}, Salary={salary}")

    def details(self, dept):
        print(f"Company Method: Department = {dept}")

class Employee(Person, Company):
    def __init__(self, name, age, company_name, salary):
        print("Employee Constructor Start")
        super().__init__(name, age)              # Goes to Person first
        Company.__init__(self, company_name, salary)  # Manually call Company
        print("Employee Constructor End")

    def details(self, city, dept):
        print("Employee Method Start")
        super().details(city)          # Calls Person's details
```

```
        Company.details(self, dept)    # Calls Company details
        print("Employee Method End")

e = Employee("John", 30, "TechCorp", 75000)
e.details("Bangalore", "IT")
```

4.hierarchical inheritance:

```
**************************
*A multiple child class inheriting property from
a single parent class is called hierarchical
inheritance.
diagram:

"""
class Parent:
    def home(self):
        print("Parent Home")

class Child1(Parent):
    age = 18
# print(dir(Child1))       ['age', 'home']
# c = Child1()
# print(c.age)
```

```
# c.home()

# 18
# Parent Home

class Child2(Parent):
   def job(self):
       print("searching for job")
# print(dir(Child2))      ['job', 'home']
# c1 = Child2()
# c1.job()
# c1.home()
# searching for job
# Parent Home
```

Example 02

```
# Parent class
class Person:
   def display(self):
       print("I am a person")
```

```
# Child class 1
class Student(Person):
    def study(self):
        print("I am a student and I study")

# Child class 2
class Teacher(Person):
    def teach(self):
        print("I am a teacher and I teach")

# Creating objects
s = Student()
t = Teacher()

# Accessing parent method from both children
s.display()
s.study()
```

```
t.display()
t.teach()
```

Example 03 With `__init__()` Constructor

```
class Company:
    def __init__(self, name):
        self.company_name = name

    def show_company(self):
        print(f"Company Name: {self.company_name}")

class Employee(Company):
    def __init__(self, name, emp_name, emp_id):
        super().__init__(name)
```

```
        self.emp_name = emp_name
        self.emp_id = emp_id

    def show_employee(self):
        print(f"Employee: {self.emp_name}, ID: {self.emp_id}")

class Manager(Company):
    def __init__(self, name, mgr_name, dept):
        super().__init__(name)
        self.mgr_name = mgr_name
        self.dept = dept

    def show_manager(self):
        print(f"Manager: {self.mgr_name}, Department: {self.dept}")

# Creating objects
```

```
e = Employee("TechCorp", "Alice", 101)
m = Manager("TechCorp", "Bob", "HR")

e.show_company()
e.show_employee()

m.show_company()
m.show_manager()
```

Example 04

Hierarchical with Method Overriding

```
class Vehicle:
    def start(self):
        print("Vehicle is starting")

class Car(Vehicle):
```

```
    def start(self):  # overriding parent method
        print("Car is starting with key ignition")

class Bike(Vehicle):
    def start(self):  # overriding parent method
        print("Bike is starting with self-start button")

c = Car()
b = Bike()

c.start()  # overridden in Car
b.start()  # overridden in Bike
```

5.hybrid inheritance:

```
**********************
*A combination of 2 or more inheritance is called
hybrid inheritance.
```

```python
class A:
    a = 10

class B(A):
    b = 20

class C(A):
    c = 30

class D(B, C):
    d = 40

#single level inheritance
# d = D()
# print(d.b)          #20
# print(d.d)          #40

#multi level inheritance
# d = D()
# print(d.a)            #10
# print(d.b)            #20
# print(d.d)            #40

#multiple inheritance
# d = D()
# print(d.a)            #10
# print(d.b)            #20
# print(d.d)            #40
```

```
#hierarchical inheritance
# c = C()
# print(c.a)          #10
# print(c.c)          #30

# b = B()
# print(b.a)          #10
# print(b.b)          #20
```

Example 02
Hybrid (Hierarchical + Multiple)

```
# Base class
class A:
    def featureA(self):
        print("Feature A from class A")

# Hierarchical inheritance
class B(A):
    def featureB(self):
        print("Feature B from class B")

class C(A):
    def featureC(self):
        print("Feature C from class C")
```

```
# Multiple inheritance (B and C → D)
class D(B, C):
    def featureD(self):
        print("Feature D from class D")

# Create object of D
obj = D()
obj.featureA()   # from A
obj.featureB()   # from B
obj.featureC()   # from C
obj.featureD()   # from D
```

Example 03

Hybrid (Single + Multilevel + Multiple)

```
# Single inheritance
class School:
    def school_info(self):
        print("This is XYZ School")

# Multilevel inheritance
class Teacher(School):
    def teacher_info(self):
```

```
        print("I am a teacher in XYZ
School")

class Student(Teacher):
    def student_info(self):
        print("I am a student of XYZ
School")

# Another independent class
class Sports:
    def sports_info(self):
        print("I play football for the
school team")

# Hybrid: Multiple inheritance using
Student + Sports
class Scholar(Student, Sports):
    def scholar_info(self):
        print("I am a scholar student")

# Object of Scholar
s = Scholar()
s.school_info()   # from School
s.teacher_info()  # from Teacher
s.student_info()  # from Student
s.sports_info()   # from Sports
s.scholar_info()  # from Scholar
```

Example 04

Hybrid with ***super()*** *& Method Resolution Order (MRO)*

```
class A:
    def show(self):
        print("A class method")

class B(A):
    def show(self):
        print("B class method")
        super().show()

class C(A):
    def show(self):
        print("C class method")
        super().show()

class D(B, C):   # Multiple inheritance
    def show(self):
        print("D class method")
        super().show()

obj = D()
```

```
obj.show()
```

Extra Information

```
#example on hierarchical inheritance
class RBI:
    def money_back(self):
        print("amount will be double if u fixed
for 5 Years")

class Icici(RBI):
    def FD_account(self):
        print("rate of interest is 7.5%")

class Postoffice(RBI):
    def saving_scheme(self):
        print("save money")

# p = Post Office()
# p.money_back()
# p.saving_scheme()
# amount will be double if u fixed for 5 Years
# save money

# i = Icici()
# i.money_back()
# i.FD_account()
```

```
# amount will be double if u fixed for 5 Years
# rate of interest is 7.5%
```

"""

Method overriding:

---

*developing a method in subclass with the same name and signature(variables/arguments) in super class, but different
implementation in subclass is called method overriding.

*when we do method overriding we will get a subclass implementation, that is overridden implementation.

*when we want subclass implementation along with superclass implementation, then we use super().

*when we want subclass implementation along with superclass implementation then we use
super() that is called "method chaining".
syntax:

---

super().method-name()

constructor chaining/overriding:

```
*******************************
*calling subclass constructor along with
super-class constructor then we use super(), that
is
called "constructor chaining".
syntax:
-------
super().__init__()

*in constructor chaining sub-class(child) is the
responsible to full-fill the super-class(parent)
class constructor.

*means if parent class constructor is accepting
any argument then, in child class we need to
pass the argument and call the constructor.

super():
-------
*super() calling statement will be used only in
the inheritance concept.

*this is used to to call super class
property/members inside the sub-class.
"""
#variable overriding
# a = 100
# b = 20
# print(a)        #100
```

```python
# a = 200
# print(a)          #200

#function overriding
# def add():
#     print("addition 1")
# def add():
#     print("addition 2")
#
# add()             #addition 2

#example on class variable riding
class OLA_bike:          #super/base/parent class
    price = "1.5lac"
    def features(self):
        print("Good electrical vehicle")

class Dealer1(OLA_bike):    #sub/derived/child
class
    price = "1.55Lac"
    def red_color(self):
        print("im a red color")

# d = Dealer1()
# print(f"price of bike is {d.price}")
# price of bike is 1.55Lac

#example on class variable overriding and also
modification
class Demo:
    a = 10
```

```
class Sample(Demo):
    a = 20

# s = Sample()
#before modification
# print(s.a)            #20
#after modification
# s.a = 30
# print(s.a)          #30



#example on method overriding
class Test:
    def fun(self):
        print("Test class fun method")

class Test1(Test):
    def fun(self):
        print("Test1 class fun method")

# t = Test1()
# t.fun()
# Test1 class fun method

#example on method chaining
class Test:
    def fun(self):
        print("Test class fun method")
```

```
class Test1(Test):
    def fun(self):
        super().fun()
        print("Test1 class fun method")

# t = Test1()
# t.fun()
# Test class fun method
# Test1 class fun method

#example on method overriding, but different
signature in subclass(we get error)
class Dim:
    def add(self):
        print("addition")

class Sim:
    def add(self, a):
        print("addition2")

# s = Sim()
# s.add()
# Sim.add() missing 1 required positional
argument: 'a'


#example on constructor overriding
class Sample:
    def __init__(self):
        print("im a parent constructor")
```

```
class Simple(Sample):
    def __init__(self):
        print("im a child constructor")

# s = Simple()
# i'm a child constructor

#example on constructor chaining
class Sample:
    def __init__(self):
        print("im a parent constructor")

class Simple(Sample):
    def __init__(self):
        super().__init__()
        print("im a child constructor")

# s = Simple()
# i'm a parent constructor
# i'm a child constructor
```

Example in method overriding:---->

example->01 before without super()

```
class Test:
    def spam(self):
        print("Hello")

class Best(Test):
    def spam(self):
        print("bye")

b=Best()
b.spam()
```

with super()function

```
 class Test:
     def spam(self):
         print("Hello")

 class Best(Test):
     def spam(self):
         super().spam()
         print("bye")

 b=Best()
 b.spam()
```

example-->02

```
class Sample:
    def __init__(self):
        print("its constructor class")


class Demo(Sample):
    def __init__(self):
        super().__init__()
        print("its Demo class")

d=Demo()
```

example-->03

```
class Bank:
    def
__init__(self,cname,Branch,IFSC,location):
        self.cname=cname
        self.Branch=Branch
        self.IFSC=IFSC
        self.location=location
```

```
    def Customer_Deatils(self):
        print(customer name is {self.cname} and
bank branch name is {self.Branch} and bank IFsC
code is {self.IFSC} and bank location is
{self.location}')


class ICIC_BANK(Bank):
    def __init__(self,pan card number,aadhar
number):

super().__init__(cname="Naman",Branch="Delhi",IFS
C=100,location="Metro")
        self.pan card number=pan card number
        self.aadhar number=aadhaar number


    def Customer_verification(self):
        print(f' customer pan card number is
{self.pan card number} and aadhar number is
{self.aadhar number}')

i=ICIC_BANK("CSJ56PK67",12345676)
i.Customer_Deatils()
i.Customer_verification()
```

(OR)

```
class Bank:
```

```
    def
__init__(self,cname,Branch,IFSC,location):
        self.cname=cname
        self.Branch=Branch
        self.IFSC=IFSC
        self.location=location

    def Customer_Deatils(self):
        print(customer name is {self.cname} and
bank branch name is {self.Branch} and bank IFsC
code is {self.IFSC} and bank location is
{self.location}')


class ICIC_BANK(Bank):
    def
__init__(self,cname,Branch,IFSC,location,pan card
number,aadhar number):

super().__init__(cname,Branch,IFSC,location)
        self.pan card number=pan card number
        self.aadhar number=aadhaar number


    def Customer_verification(self):
        print(f' customer pan card number is
{self.pan card number} and aadhar number is
{self.aadhar number}')

i=ICIC_BANK("Naman","ICICBranch","1234
ICICI","Metro","CSJ5678PK","1234-4567-9876")
i.Customer_Deatils()
```

```
i.Customer_verification()
```

```
#constructor chaining with arguments

class Flipkart:
    def __init__(self, Pname, price):
        self.pname = Pname
        self.price = price

    def order(self):
        print(f"Ordered Product {self.pname} with
Rs{self.price}")

class Ekart(Flipkart):
    def __init__(self,pname,dboy):
        super().__init__(Pname, 3000)
        print(f"The Product {Pname} will
delivered by {dboy}")
        #self.order()        #super().order()

# e = Ekart("Watch", "Amith")
# e.order()


# The Product Watch will delivered by Amith
# Ordered Product Watch with Rs3000
```

"""

*when ever super() is calling parent class method/constructor child class is the Responsible to full fill the
argument of super class method/constructor.
"""

method overriding in multiple inheritance:

---

*method overriding in multiple inheritance MRO will take place.

*MRO(method resolution order)

*MRO is the order followed to look up for an attribute/member in classes.

*in multiple inheritance MRO will take place left to right (------>)

*to display its attribute we should write "classname.__mro__"

*This MRO process will happen internally.

```
note:
----
The MRO defines a consistent order for method
lookup to resolve potential conflicts that may
arise when multiple parent classes
define methods with the same name. This order
ensures that the methods are invoked in a
predictable and unambiguous manner.
"""

class X:
    a = 10
    b = 20
class Y:
    c = 30
    b = 50
#---------->
class Z(X, Y):
    a = 60
    c = 70
    d = 80
z = Z()

# print(Z.__mro__)
# (<class '__main__.Z'>, <class '__main__.X'>,
<class '__main__.Y'>, <class 'object'>)

# print(z.d)                #80
# print(z.a)                #60
# print(z.b)                #20
# print(z.c)                #70
```

```
# print(z.e)                #AttributeError: 'Z'
object has no attribute 'e'

"""
note:
----
*Each and every class will inherit from "object"
class, "object class" is the super most class in
python.

*when ever we are search any method it will
search in its class, if it's not found then it
will search in
object class, if it's not able to find even in
object class then it will throw "Attribute
Error".


# l = []
# l.lower()

"Attribute Error", Because it is not a method of
list class and also object class.
"""



#example on multiple inheritance
class parent1:
    def demo(self):
        print("parent1")
```

```
class parent2:
    def demo(self):
        print("parent2")

class child(parent1, parent2):
    def demo1(self):    #demo()     demo()
        print("child")

# c = child()
# c.demo1()
#child
```

```
#example on multiple inheritance with method
overriding
class parent1:
    def demo(self):
        print("parent1")

class parent2:
    def demo(self):
        print("parent2")

class child(parent1, parent2):
    def demo1(self):    #demo()     demo()
        print("child")

# c = child()
# c.demo()
```

```
#parent1
# print(child.__mro__)
# (<class '__main__.child'>, <class
'__main__.parent1'>, <class '__main__.parent2'>,
<class 'object'>)


#all 3 class same method
class parent1:
    def demo(self):
        print("parent1")

class parent2:
    def demo(self):
        print("parent2")

class child(parent1, parent2):
    def demo(self):    #demo()     demo()
        print("child")

# c = child()
# c.demo()
# child

class Vehicle:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color
    def display(self):
        print(f"The Brand is {self.brand} and is
color is {self.color}")
```

```
class Car(Vehicle):
    def __init__(self,brand, color, model):
        self.model = model
        super().__init__(brand, color)
    def display(self):
        super().display()
        print(f"The model is {self.model}")

# c = Car("Kia","white","k-22")
# c.display()
```

note:
---
*In inheritance if a parent class constructor/method requires any argument,then the child class is responsible to fully fill the argument.

COMPOSITION(HAS-A-RELATIONSHIP)

By using ClassName or By creating object we can access members of one class inside
Another class is nothing but composition.

Instead of inheriting from a class, we **create an object of one class inside another class.**

The advantage of Has-A-Relationship is code reusability.

Example 01

```
# Class 1: Engine
class Engine:
    def start(self):
        print("Engine started...")

    def stop(self):
        print("Engine stopped...")

# Class 2: Car uses Engine (Composition)
class Car:
    def __init__(self, brand):
        self.brand = brand
```

```
        self.engine = Engine()    # Composition:
Car HAS an Engine

    def drive(self):
        print(f"Driving {self.brand} car")
        self.engine.start()

    def park(self):
        print(f"Parking {self.brand} car")
        self.engine.stop()

# Using Car class
c = Car("Tesla")
c.drive()
c.park()
```

Example 02

```
# Address class
class Address:
```

```
    def __init__(self, city, state, country):
        self.city = city
        self.state = state
        self.country = country

    def show_address(self):
        return f"{self.city}, {self.state}, {self.country}"

# Person class uses Address
class Person:
    def __init__(self, name, age, address):
        self.name = name
        self.age = age
        self.address = address   # Composition

    def show_details(self):
        print(f"Name: {self.name}")
        print(f"Age: {self.age}")
```

```
        print(f"Address:
{self.address.show_address()}")


# Creating Address object separately

addr = Address("Bengaluru", "Karnataka",
"India")


# Passing it into Person

p = Person("Prince", 25, addr)

p.show_details()
```

Example 03

Composition vs Inheritance

```
# Inheritance Example

class Animal:

    def make_sound(self):

        print("Animal making sound")
```

```
class Dog(Animal):   # Inheritance (is-a)
    def bark(self):
        print("Dog barks")

d = Dog()
d.make_sound()
d.bark()

# Composition Example
class Engine:
    def start(self):
        print("Engine starts")

class Bike:   # Bike has-a Engine
    def __init__(self):
        self.engine = Engine()

    def run(self):
```

```
        self.engine.start()

        print("Bike is running")


b = Bike()

b.run()
```

POLYMORPHISM :

poly : many
morphs : forms

Defn :

An object showing different behavior at different stages
of its life cycle is called as Polymorphism

* method overloading & method overriding are the examples
for Polymorphism

In python we can achieve Overriding but we cannot achieve
Overloading

Related to Polymorphism the following topics will be discussed

* Overloading
    * method overloading
    * constructor overloading

* Overriding
    * method overriding
    * constructor overriding

```
'''
Overloading :
same name but with different number of arguments is called overloading

'''

#Operator Overloading

#1 '+' Operator
#a. sum
# print(34+45)   #79
```

```
# print(25.5 + 30.8)  #56.3


#
# #b.concatenation
# print('hello ' + 'world ' + 'universe')
#hello world universe
# print('Take ' + 'care ' + 'yourself')   #Take
care yourself


#
# #c. merging
# print([2, 4, 6, 8] + [3, 6, 9, 12]) #[2, 4, 6,
8, 3, 6, 9, 12]
# print((1, 3, 5, 7) + (4, 8, 12, 16))  #(1, 3,
5, 7, 4, 8, 12, 16)



#2. '*' Operator
#a.product
# print(3*5)   #15
# print(3.8 * 2.9)  #11.02
#


# #b.replicating
# print('hello'*3)   #hello hello hello
#
```

```
# #c.unpacking
# print(*[2, 4, 6, 8], *[1, 3, 5, 7], *(3, 6, 9,
12))   #2 4 6 8 1 3 5 7 3 6 9 12

'''

Method overloading is not possible
'''

# class Sample:
#
#     def add(self,a):
#         print(a)
#
#     def add(self,a,b):
#         print(a,b)
#
#     def add(self,a,b,c):
#         print(a,b,c)

# s = Sample()
# s.add(3)  #TypeError: Sample.add() missing 2
required positional arguments: 'b' and 'c'
# s.add(3, 5)


#Not a valid justification

# class Test:
#
#     def t1(self):
#         print('no arguments')
```

```
#
#     def t2(self,a):
#         print('1 arguments')
#
#     def t3(self,a,b):
#         print('2 arguments')
#
# t = Test()
# t.t1()  #no arguments
# t.t2(3)  #1 arguments
# t.t3(3,6)   #2 arguments


#50 % Justification
#Default Parameters

# class Test:
#
#     def sum(self,a=0,b=0,c=0,d=0):
#         print(a+b+c+d)
#
# t = Test()
# t.sum()   #0   #no arguments
# t.sum(2)  #2   #1 argument
# t.sum(3,7)  #10   #2 argument
# t.sum(4,8,20)   #32  #3 argument
# t.sum(2,4,6,8)  #20   #4 arguments


'''
Constructor Overloading:
-----------------------
```

```
Developing multiple constructor in same class
with variation
in argument list is called as Constructor
overloading
'''
# class Demo:
#
#     def __init__(self):
#         print('constructor-1')
#
#     def __init__(self,a):
#         print('constructor-2')
#
#     def __init__(self,a,b):
#         print('constructor-3')
#
# d = Demo(10,20)   #constructor-3


#By default Parameters
#
# class Demo:
#
#     def __init__(self):
#         print('constructor-1')
#
#     def __init__(self,a):
#         print('constructor-2')
#         print(a)
#
#
#     def __init__(self,a=0,b=0):
```

```
#         print('constructor-3')
#         print(a+b)
# #
# d = Demo()
'''
o/p:
constructor-3
10
'''


#Method Overriding

# class University:
#
#     def internals(self):
#         print('Internals exams is scheduled on
30-06-2023')
#
# class College(University):
#
#     def intervals(self):
#         print('Internals Scheduled on
05-07-2023')
#
# c = College()
# c.intervals()   #Internals Scheduled on
05-07-2023
```

Operator Overloading in Python

1. Definition

- Operator overloading means **using the same operator** (like `+`, `-`, `*`, `==`, etc.) for **different purposes** depending on the operands.

- Python allows us to **redefine how operators work** with user-defined classes using **special (magic) methods**.

Example:

```
print(10 + 20)          # Addition
print("Hi" + "Bye")     # String concatenation
print([1,2] + [3,4])    # List merge
```

Here, `+` behaves differently → **this is operator overloading.**

Special (Magic) Methods for Overloading

These are **built-in dunder methods** (start and end with `__`).

Some common ones:

Operator	Magic Method
+	__add__(self, other)
-	__sub__(self, other)
*	__mul__(self, other)
/	__truediv__(self, other)
//	__floordiv__(self, other)
%	__mod__(self, other)
**	__pow__(self, other)

ex-1

```
class Book:
    def __init__(self, pages):
        self.pages = pages
```

```
    def __add__(self, other):
        return self.pages + other.pages

b1 = Book(100)
b2 = Book(200)
print(b1 + b2)   # 300
```

Overloading Comparison Operators

```
class Student:
    def __init__(self, marks):
        self.marks = marks

    def __gt__(self, other):   # Greater than
        return self.marks > other.marks

s1 = Student(85)
s2 = Student(90)

print(s1 > s2)  # False
```

Overloading `__str__` for Readable Output

```
class Complex:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Complex(self.x + other.x, self.y
+ other.y)

    def __str__(self):
        return f"{self.x} + {self.y}i"

c1 = Complex(2, 3)
c2 = Complex(4, 5)

print(c1 + c2)
```

Note:-->

. Key Differences: Function vs Method

Feature	Function	Method
Definition Place	Defined outside a class	Defined inside a class
First Argument	No default `self`	Always takes `self` as first parameter (instance reference)
Call Style	`function_name(args)`	`object.method(args)`
Binding	Independent	Bound to an object
Example	`len([1,2,3])`	`[1,2,3].append(4)`

Every method is a function, but not every function is a method.

A **method is just a function defined inside a class and bound to objects**.

Duck Typing in Python

Definition

- **Duck Typing** is a concept in Python where the **type or class of an object is less important** than the methods or properties it has.

- In simple words: *“If it looks like a duck, swims like a duck, and quacks like a duck, then it is treated as a duck.”*

- Python checks for **behavior (methods/attributes)**, not the actual type of the object.

ex-->1

```
class Dog:
    def sound(self):
        print("bark......")

class Cat:
    def sound(self):
        print("Meow......")

class Duck:
    def sound(self):
        print("Quack.......")

def Animal(x):
    x.sound()

# l=[Dog(),Cat(),Duck()] #l=Dog(),l=Cat(),l=Duck()

for i in (Dog(),Cat(),Duck()):
    Animal(i)
```

```
'''

'''
ex--->2
class Bird:
    def fly(self):
        print("am flying")
class Plane:
    def fly(self):
        print("am travelling")
class Human:
    def fly(self):
        print("am plying above------>****")
def Check(q):
    http://q.fly()

for i in (Bird(),Plane(),Human()):
    Check(i)

print()
l=[Bird(),Plane(),Human()]
for k in l:
    Check(k)

print()

x=Bird()
y=Plane()
```

```
z=Human()
Check(x)
Check(y)
Check(z)
```

Encapsulation:

***Encapsulation refers to the binding of data and code together into a single unit is called as encapsulation**

***declaring a member of class with private and restrict the access outside the class, provide**
The class indirectly by using getters and setters is called encapsulation.

***in python we can't achieve 100% encapsulation**

***in python we have 3 access specifier**

1.public

2.private

3.protected

1. Public access specifier:-
---
*any member(variable/method) without any prefix(_) then it is called as public access specifier.

*it can be accessed within the class and outside the class(any where)

```
# ex:- var_name, def add()
"""
"""
class Employee:
    def __init__(self,name,pay):
        self.name = name
        self.pay = pay

    def disp(self):
        print("in display")
```

```
e = Employee("amith", 100)
print(e.name)   # o/p:- smith
e.disp()    #in display



"""
2. Protected access specifier
------------------------------
*any member which is starting/prefixed with
single underscore(_) then it is called as
protected.

*ex:-   _variablename, def _disp()
"""
class Emp:
    _pin = 3245
    def _disp(self):
        print("in display")

# e = Emp()
# print(e._pin)       #3245
# e._pin = 3322
# print(e._pin)       #3322

class Emp:
    def __init__(self, name, pay):
        self._name = name
```

```
        self._pay = pay

    def disp(self):
        print("in display")

# e = Emp("amith", 100)

# print(e.name)  # o/p: Attribute Error Bcz
its in _name

# print(e.__dict__)  #o/p:- {
_name_:'amith', _pay_:100,.......}

# print(e._name)

 #here we got to know the attribute name,
but actually we should access outside the
class, so that's why in python

# we can't achieve complete encapsulation.
so a programmer it is a conversion that if
it in _var then it is a
protected. we can access outside the class
but we can't access


# the proper example of encapsulation by
using getters() and setters() methods
```

```
# no rule of getters() and setters() we can
write any name, but as per standard we use
these names.

class Emp:
    _pin = 3245
    def _disp(self):
        print("in display")

    def getters(self):
        return self._pin

    def setters(self, pin):
        self._pin = pin

# e = Emp()
# pin = e.getters()
# print(pin)
# e.setters(4444)
# pin = e.getters()
# print(pin)

"""
class Emp:
    def __init__(self, name, pay):
        self._name = name
        self._pay = pay
```

```
    def get_name(self):                 #similar
to getters()
        return _name

    def get_pay(self):                  #similar
to getters()
        return _pay

    def set_name(self, name):           #similar
to setters()
        _name = name

    def set_pay(self, pay):             #similar
to setters()
        _pay = pay

e = Emp("abhi", 1000)
print(e.get_name())  #accessing protected
variable outside the class indirectly
print(e.get_pay())

e.set_name("appu") # setting protected
variable from outside class
e.set_pay("9000")
print(e.get_name())
print(e.get_pay())
```

```
"""
# even we can make as function as private
and protected, access outside class take
another method and return the private method
then store
# in other variable call




# 3. Private access specifier


# any member which is starting/prefixed with
double underscore(__) then it is called as
private

# ex: -   __variablename, def __disp()

# private variables will be stored in
(_ClassName__variableName) format only.

# if we want to access any private variable
then below format.

# obj._clasname__variablename (or)
classname._classname__variablename
"""
```

```
class Emp:
    def __init__(self, name, pay):
        self.__name = name
        self.__pay = pay

    def display(self):
        print("display")

e = Emp("apu", 1000)

print(e.__dict__)  # o/p- {'_Emp__name':
'apu', '_Emp__pay': 1000}

#print(e.__name) # AttributeError: 'Emp'
object has no attribute '__name'
                # Bcz by default it will be
written as _Emp__name so

# print(e._Emp__name)  # o/p:- Appu ==> we
will get output but it is not correct Bcz we
are not knowing so it is encapsulated.
"""



# Encapsulation in inheritance:-
```

```
# accessing protected variable in another
class or child class

class Parent:
    _a = 10

    def disp(self):
        print(self._a)

class Child(Parent):
    def get_a(self):
        print(self._a)

# c = Child()
# c.get_a()  # o/p: 10




# accessing private variable in another
class or child class

class Parent:
    __a = 10

    def disp(self):
        print(self.__a)
```

```
class Child(Parent):
    def get_a(self):
        print(self.__a)

# c = Child()

# c.get_a()  # o/p: attribute error Bcz all
private variable will in   (_Parent__a)
           # until and unless we the name in
which it is stored we can't access so we
can't access the private variable
          # in another class.




# overriding protected variable

class Parent:
    _a = 10

    def disp(self):
        print(self._a)

class Child(Parent):
    _a = 20

    def get_a(self):
```

```
        print(self._a)
        print(super()._a)

# c = Child()
# c.get_a()  # o/p: 20  # we will get
overridden implementation




# overriding private variable

class Parent:
    __a = 10

    def disp(self):
        print(self.__a)

class Child(Parent):
    __a = 20

    def get_a(self):
        print(self.__a)
        # print(super().__a) even we can't
access the parent class __a bcz it will in
_class_var format

# c = Child()
```

```
# #print(c.__a)  # error, can't outside
class
# c.get_a() # o/p 20


# note:-
"""
*we can access protected variables in child
class.

*we can override the protected variable in
child class

*we can't access private variable in child
class

*we can't override the private variable in
child class(not at all possible Bcz variable
will store with _class__variable
"""



# property decorator

"""
```

The @property decorator is a built-in decorator in Python for the property() function.
Use @property decorator on any method in the class to use the method as a property.

You can use the following three decorators to define a property:

@property: Declares the method as a property and to get/print the value
@<property-name>.setter: Specifies the setter method for a property that sets the value to a property.
@<property-name>.deleter: Specifies the delete method as a property that deletes a property.
"""

```
"""
class Emp:
    def __init__(self, name, pay):
        self.name = name
        self.__pay = pay   #private
variable

    def get_pay(self):
```

```
        print("getting apy")
        return self.__pay

    def set_pay(self,pay):
        print("setting apy")
        self.__pay = pay

    def del_pay(self):
        print("deletting apy")
        del self.__pay

e = Emp("appu", 2000)
print(e.get_pay())
e.set_pay(100)
print(e.get_pay())
"""
"""
class Emp:
    def __init__(self, name, pay):
        self.name = name
        self.__pay = pay   #private
variable

    @property
    def get_pay(self):
        print("getting apy")
        return self.__pay
```

```
    @get_pay.setter
    def set_pay(self,pay):    #we can add
raise error we can restrict of setting
private variable
        print("setting apy")
        self.__pay = pay

    @set_pay.deleter    #@get_pay.deleter
    def del_pay(self):
        print("deletting apy")
        del self.__pay


e = Emp("appu",2000)
print(e.get_pay)
e.set_pay = 4000
print(e.get_pay)
e.del_pay
"""
"""
class Emp:
    def __init__(self, name, pay):
        self.name = name
        self.__pay = pay   #private
variable
```

```
    @property
    def pay(self):
        print("getting apy")
        return self.__pay

    @pay.setter
    def pay(self,pay):
        print("setting apy")
        self.__pay = pay

    @pay.deleter
    def pay(self):
        print("deletting apy")
        # del self.__pay

e = Emp("appu",2000)
print(e.pay)
e.pay = 4000
print(e.pay)
del e.pay
"""


"""
# Defining class
class Portal:
```

```
    # Defining __init__ method
    def __init__(self):
        self.__name =''

    # Using @property decorator
    @property
    # Getter method
    def name(self):
        return self.__name

    # Setter method
    @name.setter
    def name(self, val):
        self.__name = val

    # Deleter method
    @name.deleter
    def name(self):
       del self.__name

# Creating object
p = Portal();

# Setting name
p.name = 'josh'

# Prints name
```

```
print (p.name)

# Deletes name
del p.name

# As name is deleted above this
# will throw an error
print (p.name)



#example:--->
class Test:
    def __init__(self,product):
        self.product=product

    @property                              #getter
method
    def getter_(self):
        return self.product


    @getter_.setter                       #setting
method
    def getter_(self,product):
        self.product=product
```

```
    @getter_.deleter                    #delete
    def getter_(self):
        del self.product

t=Test("watch")
print(t.getter_)
t.getter_=("mobile")
print(t.getter_)
del t.getter_
print(t.getter_)   #attribute error
```

ABSTRACTION

Hiding the complexity of a features and displaying only the
The required property to the end-user is called ABSTRACTION.

Abstract Method

Sometimes we don't Know about implementation,still we can declare a method,such types of methods are called

abstract method, i.e abstract method has only declaration but not implementation

In python we can declare abstract method by using @abstractmethod decorator as follows

```
@abstractmethod
Def M1(self):
    Pass
```

@abstractmethod decorator present in abc module,hence compulsory we should import abc module,otherwise we will get error

abc – abstract base class module

```
Class Test:
    @abstractmethod
    Def M1(self):
        Pass
```

Name error abstract method not defined

```
From abc import *

Class Test:
    @abstractmethod
```

```
Def M1(self):
    Pass
```

Abstract Class

Sometimes implementation of a class is not a complete,such type of partially implementation classes are called abstract classes Every abstract class in Python should be derived from ABC class which is Present in abc module

```
from abc import ABC
abc ---> module/file
ABC ---> ClassName
@abstractmethod ---> decorator
```

interface :

* a class consist of pure abstract method is called as Interface

(we don't have any concept as interface in python)

concrete method:

```
A method which is having declaration & definition
is called
as Concrete method

def add(self)                --> declaration
   print('addition')         --> definition
"""


#case--->01
 from abc import *
  class Test:
     pass

 t=Test()

#in the above code we can create object for Test
class because it is concrete class and it does
not
#contain any abstract method




#case-->02

# from abc import *
# class Test(ABC):
#     pass
#
# t=Test()
```

```
#in the above code we can create object,even it
is derived from ABC class but doesn't contain any
#abstract method




#case-->03
# from abc import *
# class Test(ABC):
#     @abstractmethod
#     def spam(self):
#         pass
#
# t=Test()
#TypeError: Can't instantiate abstract class Test
with abstract method spam



#case-->04

# from abc import *
# class Test:
#     @abstractmethod
#     def spam(self):
#         print("hi")
#
# t=Test()
# t.spam()
```

#we can create object even class contains abstract method because we are not extending ABC class

Example:--->01

```
class TEST(ABC):
    @abstractmethod
    def Subject1(self):
        pass

    @abstractmethod
    def Subject2(self):
        pass

    @abstractmethod
    def Subject3(self):
        pass


class Test2(TEST):

    def Subject1(self):
        print("my first subject is PYTHON")

    def Subject2(self):
        print("my second subject is SQL")
```

```
    def Subject3(self):
        print("my third subject is Selenium")

t=Test2()
t.Subject1()
t.Subject2()
t.Subject3()
```

example:-->02

```
from abc import ABC,abstractmethod
class Bank(ABC):
    @abstractmethod
    def Balance(self):
        pass

    @abstractmethod
    def deposit(self,amount):
        pass

    @abstractmethod
    def withdrawal(self,amount):
        pass

class ATM(Bank):
    def __init__(self,cname,bal):
        self.name=cname
        self.bal=bal
```

```
    def Balance(self):
        print(f' total available balance is
{self.bal}')


    def deposit(self,amount):
        print(f' total available balance is
{self.bal}')
        self.bal+=amount
        print(f' after depositing total amount is
{self.bal}')

    def withdrawal(self,amount):
        print(f' total remains balance is
{self.bal}')
        self.bal-=amount
        print(f' after withdrawal remains balance
is {self.bal}')

a=ATM("Ravi",10000)
a.Balance()
print()
a.deposit(5000)
print()
a.withdrawal(4000)
print()


"""
output:--->
```

```
 total available balance is 10000

 total available balance is 10000
 after depositing total amount is  15000

 total remains balance is 15000
 after withdrawal remains balance is 11000
"""


Assignment

#Book a movie ticket
```